

1. Constraint-Based Data Cleaning Using NumPy and Pandas

A patient health dataset may contain missing values, incorrect numerical values, and inconsistent categorical data. A cleaning pipeline can be designed using **Pandas and NumPy** to enforce data constraints.

Steps

1. Load and inspect the data

```
import pandas as pd
import numpy as np

df = pd.read_csv("patients.csv")

print(df.head())
print(df.isnull().sum())
```

2. Handle missing values

Missing numerical values can be replaced using the median, while categorical values can be replaced using the mode.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["BloodPressure"] = df["BloodPressure"].fillna(
    df["BloodPressure"].median()
)
df["Gender"] = df["Gender"].fillna(
    df["Gender"].mode()[0]
)
```

This ensures that no missing values remain.

3. Enforce medically valid ranges

Filtering can identify invalid values.

```
df.loc[(df["Age"] < 0) | (df["Age"] > 120), "Age"] = np.nan
df["Age"] = df["Age"].fillna(df["Age"].median())

df.loc[
    (df["BloodPressure"] < 50) |
```

```

(df["BloodPressure"] > 250),
"BloodPressure"
] = np.nan

df["BloodPressure"] = df["BloodPressure"].fillna(
    df["BloodPressure"].median()
)

```

This prevents medically impossible values from remaining in the dataset.

4. Standardize categorical fields

```

df["Gender"] = (
    df["Gender"]
    .str.strip()
    .str.lower()
)

df["Gender"] = df["Gender"].replace({
    "m": "Male",
    "male": "Male",
    "f": "Female",
    "female": "Female"
})

```

Thus, values such as **M**, **male**, and **Male** are converted into a common format.

5. Use indexing and filtering

Pandas indexing and filtering can locate records that violate constraints.

```

invalid = df[
    (df["Age"] < 0) |
    (df["Age"] > 120)
]

print(invalid)

```

This allows invalid records to be identified before correction.

6. Use aggregation

Aggregation functions can calculate statistics needed for cleaning and validation.

```

print(df["Age"].mean())
print(df["Age"].median())

```

```
print(df.groupby("Gender")["Age"].mean())
```

These functions help understand the data and choose suitable replacement values.

7. Final validation

```
print("Missing values:")  
print(df.isnull().sum())
```

```
print("Age range:")  
print(df["Age"].min(), df["Age"].max())
```

```
print("Gender values:")  
print(df["Gender"].unique())
```

Conclusion

A constraint-based cleaning pipeline uses **indexing, filtering, replacement, and aggregation** to ensure that the dataset is complete, medically valid, and standardized. Instead of deleting large numbers of records, invalid values can be corrected using reliable statistical values, thereby preserving maximum data integrity.

2. Memory-Efficient Processing of a Massive NumPy Dataset

When a numerical dataset is too large for available memory, NumPy operations should be designed to avoid unnecessary copies and temporary arrays.

1. Use efficient data types

Choose the smallest suitable data type.

```
import numpy as np
```

```
data = np.zeros(  
    10_000_000,  
    dtype=np.float32  
)
```

`float32` requires less memory than `float64`.

2. Use slicing instead of copying

```
part = data[:,10]
```

NumPy slicing normally creates a **view** rather than a complete copy, reducing memory usage.

Avoid unnecessary operations such as:

```
part = data[0:1000000].copy()
```

unless a separate copy is actually required.

3. Use in-place operations

Instead of:

```
data = data * 2
```

use:

```
data *= 2
```

The second approach avoids creating another large array.

Similarly:

```
data += 10
```

```
data -= 5
```

```
data /= 2
```

4. Process data in chunks

For extremely large data, process smaller portions at a time.

```
chunk_size = 1_000_000
```

```
for start in range(0, len(data), chunk_size):
```

```
    end = start + chunk_size
```

```
    chunk = data[start:end]
```

```
    chunk *= 2
```

Only a small portion needs to be processed at a time.

5. Efficient random sampling

Instead of creating a large random array, generate only the required sample.

```
sample_index = np.random.choice(
    len(data),
    size=10000,
    replace=False
)
```

```
sample = data[sample_index]
```

6. Optimize aggregation

NumPy aggregation functions are efficient:

```
mean = np.mean(data)
maximum = np.max(data)
minimum = np.min(data)
total = np.sum(data)
```

```
print(mean)
print(maximum)
print(minimum)
print(total)
```

For very large datasets, calculate statistics chunk-by-chunk instead of creating additional arrays.

7. Optimize indexing

Use simple slicing whenever possible:

```
data[:100000]
data[:,2]
```

Avoid creating multiple large temporary arrays through complicated indexing.

8. Debug memory and performance problems

Check memory usage:

```
print(data.nbytes / (1024 ** 2), "MB")
```

Check execution time:

```
import time
```

```
start = time.time()
```

```
result = np.mean(data)
```

```
print("Time:", time.time() - start)
```

If memory usage is too high:

- Use `float32` instead of `float64` where appropriate.
- Process the dataset in chunks.
- Use slicing instead of unnecessary copies.
- Use in-place operations.
- Store only required columns/values.
- Delete unused arrays using `del`.

```
del unused_array
```

Conclusion

For massive NumPy datasets, **efficient data types, slicing, in-place operations, chunk processing, optimized indexing, and aggregation** help reduce memory consumption. Monitoring `nbytes` and execution time makes it easier to identify memory and performance bottlenecks while maintaining efficient processing.